

Module #3 Introduction to OOPS Programming

Name = Komal Nafde

1. What are the key differences between Procedural Programming and Object-Oriented Programming (OOP)?

Ans-

Basis	Procedural Programming	Object-Oriented Programming
Approach	Top-down approach	Bottom-up approach
Main Focus	Functions and procedures	Objects and classes
Data Security	Less secure; data can be accessed easily	More secure due to encapsulation
Reusability	Limited	High through inheritance
Maintenance	Difficult for large programs	Easier to maintain and update
Real-World Modeling	Not suitable	Suitable for world problem
Examples	C, Pascal	Java, C++, Python

2. List and explain the main advantages of OOP over POP?

Ans-

Main Advantages of Object-Oriented Programming (OOP) over Procedural-Oriented Programming (POP)

Object-Oriented Programming (OOP) offers several advantages over Procedural-Oriented Programming (POP). These advantages make OOP more suitable for developing large, complex, and maintainable software applications.

1. Code Reusability

OOP supports **inheritance**, which allows one class to reuse the properties and methods of another class. This reduces code duplication and saves development time.

2. Data Security

OOP uses **encapsulation**, which hides data inside objects and allows access only through methods. This protects data from unauthorized modification.

3. Modularity

Programs are divided into independent classes and objects. Each class handles a specific responsibility, making the code easier to understand and manage.

4. Easy Maintenance

Changes made in one class usually do not affect other classes. This makes it easier to debug, update, and maintain the program.

5. Flexibility

OOP supports **polymorphism**, which allows the same method name to perform different tasks depending on the object.

6. Real-World Modeling

OOP represents real-world entities as objects, making it easier to design and solve practical problems.

7. Scalability

OOP is well suited for large projects because new features can be added by creating new classes without changing existing code significantly.

8. Improved Productivity

Because of reusability, modularity, and easier maintenance, development becomes faster and more efficient.

3. Explain the steps involved in setting up a C++ development environment?

Ans-

Steps to Set Up a C++ Development Environment

To write, compile, and run C++ programs, you need to set up a C++ development environment. The main steps are as follows:

1. Install a C++ Compiler

A compiler converts C++ source code into an executable program.

Common C++ compilers include:

- GCC
- Clang

- Microsoft Visual C++

2. Install an Integrated Development Environment (IDE) or Code Editor

An IDE provides tools for writing, compiling, and debugging programs.

Popular options:

- Visual Studio Code
- Code::Blocks
- Dev-C++
- Visual Studio

3. Configure the Compiler

Connect the compiler with your IDE/editor.

For example:

- In Visual Studio Code, install the C/C++ extension and set the compiler path.
- In Code::Blocks or Dev-C++, the compiler is often included.

4. Create a New C++ Source File

Create a file with the .cpp extension, such as main.cpp.

5. Write a Simple Program

Example:

```
#include <iostream>
using namespace std;

int main() {
    cout << "Hello, World!";
    return 0;
}
```

6. Compile the Program

Use the compiler to convert the source file into an executable.

Example with GCC:

```
g++ main.cpp -o main
```

7. Run the Program

Execute the compiled program.

- On Windows:

main.exe

- On Linux/macOS:

./main

8. Test and Debug

Fix any syntax or logical errors using the IDE's debugging tools.

4. What are the main input/output operations in C++? Provide examples

Ans-

Main Input/Output Operations in C++

Input and output (I/O) operations in C++ are used to read data from the user and display results on the screen. These operations are provided by the `<iostream>` header file.

1. Output Operation (cout)

cout (character output) is used to display data to the console.

Example:

```
#include <iostream>
using namespace std;

int main() {
    cout << "Hello, World!";
    return 0;
}
```

Output:

Hello, World!

2. Input Operation (cin)

cin (character input) is used to take input from the user.

Example:

```
#include <iostream>
using namespace std;

int main() {
    int age;
    cin >> age;
    cout << "Age = " << age;
    return 0;
}
```

3. End Line (endl)

endl inserts a new line and flushes the output buffer.

Example:

```
cout << "Welcome to C++" << endl;
cout << "Programming";
```

4. Error Output (cerr)

cerr is used to display error messages.

Example:

```
cerr << "Error: Invalid input!";
```

5. Log Output (clog)

clog is used to display logging or debugging information.

Example:

```
clog << "Program is running...";
```

5. What are the different data types available in C++? Explain with examples

Ans-

Different Data Types Available in C++

Data types in C++ define the type of data that a variable can store. They help the compiler allocate memory and determine the kind of operations that can be performed on the data.

1. Basic (Primitive) Data Types

Data Type	Description	Example
int	Stores whole numbers	int age = 20;
float	Stores decimal numbers (single precision)	float price = 99.5;
double	Stores decimal numbers (double precision)	double pi = 3.14159;
char	Stores a single character	char grade = 'A';
bool	Stores true or false	bool isPassed = true;
void	Represents no value	void display();

2. Derived Data Types

These are created from basic data types.

- **Array:** int marks[5];
- **Pointer:** int *ptr;
- **Reference:** int &ref = age;

3. User-Defined Data Types

These are created by the programmer.

- **Structure (struct)**
- **Class (class)**
- **Enumeration (enum)**
- **Typedef / Using**

Examples:

```
struct Student {  
    int id;  
    char grade;  
};
```

```
enum Day { MON, TUE, WED };
```

4. Modified Data Types

Type modifiers change the size or range of data types.

- short int
- long int
- signed int
- unsigned int

Example:

```
unsigned int population = 1000000;
```

```
long int distance = 123456789;
```

6. Explain the difference between implicit and explicit type conversion in C++?

Ans-

Difference Between Implicit and Explicit Type Conversion in C++

Type conversion is the process of converting one data type into another in C++.

1. Implicit Type Conversion (Automatic Conversion)

Implicit type conversion is performed automatically by the C++ compiler when converting a smaller data type to a larger compatible data type.

Example:

```
#include <iostream>
using namespace std;
```

```
int main() {
    int num = 10;
    double value = num; // int is automatically converted to double

    cout << value;
    return 0;
}
```

Output:

10

Characteristics:

- Done automatically by the compiler.
 - No special syntax is required.
 - Usually safe because there is little or no data loss.
-

2. Explicit Type Conversion (Type Casting)

Explicit type conversion is performed manually by the programmer using a cast operator.

Example:

```
#include <iostream>
using namespace std;

int main() {
    double num = 10.75;
    int value = (int)num; // Explicit conversion

    cout << value;
    return 0;
}
```

Output:

10

Characteristics:

- Done manually by the programmer.
- Uses casting syntax such as (int) or static_cast<int>().
- May cause data loss.

7. What are the different types of operators in C++? Provide examples of each.

Ans- **Different Types of Operators in C++**

Operators in C++ are symbols used to perform operations on variables and values.

1. Arithmetic Operators

These operators perform mathematical calculations.

Operator	Description	Example
+	Addition	a + b
-	Subtraction	a - b
*	Multiplication	a * b
/	Division	a / b
%	Modulus	a % b

Example:

```
int a = 10, b = 3;  
cout << a + b; // 13  
cout << a % b; // 1
```

2. Relational Operators

Used to compare two values.

Operator	Meaning
==	Equal to
!=	Not equal to
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to

Example:

```
cout << (a > b); // 1 (true)
```

3. Logical Operators

Used to combine conditions.

Operator	Meaning
&&	Logical AND
^	
!	Logical NOT

Example:

```
cout << (a > 5 && b < 5);
```

4. Assignment Operators

Used to assign values.

Operator	Example
=	a = 10
+=	a += 5
-=	a -= 2
*=	a *= 3
/=	a /= 2

5. Increment and Decrement Operators

Operator	Meaning
++	Increase by 1
--	Decrease by 1

Example:

```
a++;  
b--;
```

6. Bitwise Operators

Operator	Meaning
&	Bitwise AND
^	Bitwise XOR
~	Bitwise NOT
<<	Left shift
>>	Right shift

7. Conditional (Ternary) Operator

Syntax:

```
condition ? expression1 : expression2;
```

Example:

```
int max = (a > b) ? a : b;
```

8. Explain the purpose and use of constants and literals in C++?

Ans- **Purpose and Use of Constants and Literals in C++**

In C++, **constants** and **literals** are fixed values that do not change during program execution. They are used to make programs more readable, reliable, and easier to maintain.

1. Literals

Literals are fixed values written directly in the source code.

Types of Literals in C++

Type	Example	Description
Integer Literal	100	Whole number
Floating-Point Literal	3.14	Decimal number
Character Literal	'A'	Single character
String Literal	"Hello"	Sequence of characters
Boolean Literal	true, false	Logical values

Example:

```
int age = 20;    // 20 is an integer literal
float pi = 3.14; // 3.14 is a floating-point literal
char grade = 'A'; // 'A' is a character literal
string name = "Komal"; // "Komal" is a string literal
bool flag = true; // true is a boolean literal
```

2. Constants

Constants are named values whose value cannot be changed after initialization.

Creating Constants

- Using const
- Using #define

Example using const

```
const double PI = 3.14159;
```

Example using #define

```
#define MAX 100
```

Purpose of Constants

1. Prevent accidental changes

2. **Improve code readability**
 3. **Make maintenance easier**
 4. **Represent fixed values** such as π , tax rate, or limits
-

Example Program

```
#include <iostream>
using namespace std;

int main() {
    const float PI = 3.14;
    float radius = 5;
    float area = PI * radius * radius;

    cout << "Area = " << area;
    return 0;
}
```

9. What are conditional statements in C++? Explain the if-else and switch statements

Ans- **Conditional Statements in C++**

Conditional statements in C++ are used to make decisions in a program. They allow the program to execute different blocks of code based on whether a condition is true or false.

The two most commonly used conditional statements are:

1. if-else
 2. switch
-

1. if-else Statement

The if-else statement checks a condition and executes one block of code if the condition is true and another block if the condition is false.

Syntax

```
if (condition) {  
    // code executed if condition is true  
}  
else {  
    // code executed if condition is false  
}
```

Example

```
#include <iostream>  
using namespace std;  
  
int main() {  
    int num = 10;  
  
    if (num > 0) {  
        cout << "Positive number";  
    } else {  
        cout << "Not positive";  
    }  
  
    return 0;  
}
```

2. switch Statement

The switch statement is used when there are multiple possible values for a single expression. It compares the expression with each case value and executes the matching block.

Syntax

```
switch (expression) {  
    case value1:  
        // code  
        break;  
    case value2:  
        // code  
        break;  
    default:
```

```
    // code if no case matches  
}
```

Example

```
#include <iostream>  
using namespace std;  
  
int main() {  
    int day = 2;  
  
    switch (day) {  
        case 1:  
            cout << "Monday";  
            break;  
        case 2:  
            cout << "Tuesday";  
            break;  
        default:  
            cout << "Invalid day";  
    }  
  
    return 0;  
}
```

10. What is the difference between for, while, and do-while loops in C++?

Ans- **Difference Between for, while, and do-while Loops in C++**

Loops in C++ are used to execute a block of code repeatedly as long as a condition is true.

The three main types are: **for loop, while loop, and do-while loop.**

1. for Loop

The for loop is used when the number of iterations is already known.

Syntax:

```
for (initialization; condition; update) {  
    // code  
}
```

Example:

```
#include <iostream>  
using namespace std;  
  
int main() {  
    for (int i = 1; i <= 5; i++) {  
        cout << i << endl;  
    }  
    return 0;  
}
```

Key Point:

- Best when the number of iterations is fixed.
-

2. while Loop

The while loop checks the condition before executing the loop body.

Syntax:

```
while (condition) {  
    // code  
}
```

Example:

```
#include <iostream>  
using namespace std;  
  
int main() {  
    int i = 1;  
  
    while (i <= 5) {  
        cout << i << endl;  
        i++;  
    }  
}
```



```
    return 0;
}
```

Key Point:

- Entry-controlled loop (condition checked first).
 - Used when number of iterations is not fixed.
-

3. do-while Loop

The do-while loop executes the code first and then checks the condition.

Syntax:

```
do {
    // code
} while (condition);
```

Example:

```
#include <iostream>
using namespace std;
```

```
int main() {
    int i = 1;

    do {
        cout << i << endl;
        i++;
    } while (i <= 5);

    return 0;
}
```

Key Point:

- Exit-controlled loop.
- Executes at least once, even if the condition is false.

11. How are break and continue statements used in loops? Provide examples.

Ans- **break and continue Statements in Loops (C++)**

In C++, **break** and **continue** are used to control the flow of loops like for, while, and do-while.

1. break Statement

The break statement is used to **terminate the loop immediately**, even if the condition is still true.

Example:

```
#include <iostream>
using namespace std;

int main() {
    for (int i = 1; i <= 10; i++) {
        if (i == 5) {
            break; // loop stops when i becomes 5
        }
        cout << i << endl;
    }
    return 0;
}
```

Output:

```
1
2
3
4
```

Key Point:

- Stops the entire loop.
 - Control moves outside the loop.
-

2. continue Statement

The continue statement is used to **skip the current iteration** and move to the next iteration of the loop.

Example:

```
#include <iostream>
using namespace std;

int main() {
    for (int i = 1; i <= 5; i++) {
        if (i == 3) {
            continue; // skips number 3
        }
        cout << i << endl;
    }
    return 0;
}
```

Output:

```
1
2
4
5
```

Key Point:

- Does not stop the loop.
- Only skips one iteration.

12. Explain nested control structures with an example.

Ans- **Nested Control Structures in C++**

Nested control structures occur when one control statement (like if, for, while, or switch) is placed inside another control statement. This helps in handling complex decision-making and repeated operations.

In simple words, **“nesting means one statement inside another.”**

1. Nested if Statement

A condition inside another condition.

Example:

```
#include <iostream>
using namespace std;

int main() {
    int age = 20;
    int marks = 75;

    if (age >= 18) {
        if (marks >= 50) {
            cout << "Eligible for admission";
        } else {
            cout << "Not eligible due to low marks";
        }
    } else {
        cout << "Not eligible due to age";
    }

    return 0;
}
```

2. Nested for Loop

A loop inside another loop. It is commonly used for patterns and tables.

Example:

```
#include <iostream>
using namespace std;

int main() {
    for (int i = 1; i <= 3; i++) {
        for (int j = 1; j <= 3; j++) {
            cout << i << " " << j << endl;
        }
    }
}
```

```
    return 0;
}
```

Output:

```
1 1
1 2
1 3
2 1
2 2
2 3
3 1
3 2
3 3
```

3. Nested while Loop

A while loop inside another while loop.

Example:

```
#include <iostream>
using namespace std;

int main() {
    int i = 1;

    while (i <= 3) {
        int j = 1;

        while (j <= 3) {
            cout << i << " " << j << endl;
            j++;
        }

        i++;
    }

    return 0;
}
```

13. What is a function in C++? Explain the concept of function declaration, definition, and calling.

Ans- **Function in C++**

A **function** in C++ is a block of code that performs a specific task. It helps in breaking a program into smaller parts, making it easier to read, reuse, and maintain.

1. Function Declaration (Prototype)

A function declaration tells the compiler about the function's **name, return type, and parameters**, without defining its body.

Syntax:

```
return_type function_name(parameters);
```

Example:

```
int add(int, int); // function declaration
```

2. Function Definition

A function definition contains the **actual code (logic)** of the function.

Syntax:

```
return_type function_name(parameters) {  
    // function body  
}
```

Example:

```
int add(int a, int b) {  
    return a + b;  
}
```

3. Function Calling

A function call is used to **execute the function**.

Syntax:

```
function_name(arguments);
```

Example:

```
#include <iostream>
using namespace std;
```

```
int add(int, int); // declaration
```

```
int main() {
    int result = add(5, 3); // function call
    cout << "Sum = " << result;
    return 0;
}
```

```
int add(int a, int b) { // definition
    return a + b;
}
```

Output:

Sum = 8

14. What is the scope of variables in C++? Differentiate between local and global scope.

Ans- **Scope of Variables in C++**

The **scope of a variable** in C++ refers to the region of the program where the variable can be accessed or used. In simple terms, it defines the **visibility and lifetime** of a variable.

There are mainly two types of scope:

1. Local scope
2. Global scope

1. Local Scope

A variable declared inside a function or block has **local scope**. It can only be used within that function or block.

Example:

```
#include <iostream>
using namespace std;

void show() {
    int x = 10; // local variable
    cout << x;
}

int main() {
    show();
    return 0;
}
```

Key Points:

- Declared inside a function or block
 - Accessible only within that function/block
 - Destroyed after function execution ends
-

2. Global Scope

A variable declared outside all functions has **global scope**. It can be accessed by any function in the program.

Example:

```
#include <iostream>
using namespace std;

int x = 20; // global variable

void show() {
    cout << x;
}

int main() {
    show();
}
```



```
    return 0;  
}
```

Key Points:

- Declared outside all functions
 - Accessible throughout the program
 - Exists until the program ends
-

Difference Between Local and Global Scope

Feature	Local Variable	Global Variable
Declaration	Inside function/block	Outside all functions
Scope	Limited to function/block	Entire program
Lifetime	Exists only during function execution	Exists throughout program
Default Value	Garbage value (if not initialized)	Automatically initialized to 0
Access	Only within same function	Any function can access

15. Explain recursion in C++ with an example?

Ans- **Recursion in C++**

Recursion in C++ is a programming technique where a function calls itself to solve a problem. The problem is broken into smaller sub-problems until a **base condition** is reached, which stops the recursion.

Key Components of Recursion

1. **Base Case** – Condition that stops the function from calling itself.
 2. **Recursive Case** – Part where the function calls itself.
-

Example: Factorial Using Recursion

Factorial of a number n is:

$$n! = n \times (n - 1) \times (n - 2) \times \dots \times 1$$

C++ Program:

```
#include <iostream>
using namespace std;

int factorial(int n) {
    if (n == 0 || n == 1) { // base case
        return 1;
    }
    return n * factorial(n - 1); // recursive call
}

int main() {
    int num = 5;

    cout << "Factorial = " << factorial(num);

    return 0;
}
```

Output:

Factorial = 120

How It Works (Step-by-Step)

For factorial(5):

```
5 * factorial(4)
  4 * factorial(3)
    3 * factorial(2)
      2 * factorial(1)
        returns 1 (base case)
```

Then results are multiplied back:

$5 \times 4 \times 3 \times 2 \times 1 = 120$

Advantages of Recursion

- Simplifies complex problems
 - Code becomes shorter and cleaner
 - Useful for problems like trees, graphs, and mathematical computations
-

Disadvantages of Recursion

- Uses more memory (stack memory)
- Can be slower than loops
- Risk of infinite recursion if base case is missing

16. What are function prototypes in C++? Why are they used?

Ans- **Function Prototypes in C++**

A **function prototype** in C++ is a declaration of a function that tells the compiler about the function's:

- Name
- Return type
- Parameters (data types only)

It does **not contain the function body (definition)**.

Syntax:

```
return_type function_name(parameter_list);
```

Example:

```
int add(int, int); // function prototype
```

Complete Example:

```
#include <iostream>
using namespace std;
```

```
int add(int, int); // function prototype
```

```
int main() {  
    cout << add(5, 3);  
    return 0;  
}
```

```
int add(int a, int b) { // function definition  
    return a + b;  
}
```

Why Function Prototypes are Used?

1. To Inform the Compiler

They tell the compiler that a function exists before it is used in the program.

2. To Allow Function Call Before Definition

You can call a function in main() even if its definition is written later in the program.

3. For Type Checking

The compiler checks whether the correct number and type of arguments are passed.

4. Improves Program Structure

It allows separating declaration and definition, making the program more organized and readable.

5. Useful in Large Programs

In big projects, prototypes are often placed in header files for better management.

17. What are arrays in C++? Explain the difference between single-dimensional and multi-dimensional arrays.

Ans- Arrays in C++

An **array** in C++ is a data structure that stores multiple values of the **same data type** in a single variable. All elements are stored in **continuous memory locations**, and each element is accessed using an index.

1. Single-Dimensional Array

A single-dimensional array stores elements in a **linear form (one row)**.

Syntax:

```
data_type array_name[size];
```

Example:

```
#include <iostream>
using namespace std;

int main() {
    int marks[5] = {10, 20, 30, 40, 50};

    cout << marks[0]; // Access first element
    return 0;
}
```

Key Points:

- Stores data in a single row
 - Uses one index (e.g., marks[2])
 - Easy to use and simple structure
-

2. Multi-Dimensional Array

A multi-dimensional array stores data in **rows and columns (like a table or matrix)**. The most common is a **2D array**.

Syntax:

```
data_type array_name[rows][columns];
```

Example (2D Array):

```
#include <iostream>
using namespace std;

int main() {
    int matrix[2][3] = {
        {1, 2, 3},
        {4, 5, 6}
    };

    cout << matrix[1][2]; // Access element
    return 0;
}
```

Key Points:

- Stores data in tabular form (rows and columns)
- Uses multiple indices (e.g., matrix[1][2])
- Useful for matrices, grids, tables

Difference Between Single-Dimensional and Multi-Dimensional Arrays

Feature	Single-Dimensional Array	Multi-Dimensional Array
Structure	Linear (one row)	Table-like (rows & columns)
Indexing	One index (a[i])	Multiple indices (a[i][j])
Complexity	Simple	More complex
Memory Usage	Less	More
Example	List of marks	Matrix, table of data
Use Case	Storing simple lists	Storing grids, matrices

18. Explain string handling in C++ with examples?

Ans- **String Handling in C++**

In C++, **string handling** means working with sequences of characters such as names, words, or sentences. C++ provides two ways to handle strings:

1. **C-style strings (character arrays)**
 2. **C++ string class (string)**
-

1. C-Style Strings (Character Array)

A C-style string is an array of characters ending with a **null character '\0'**.

Example:

```
#include <iostream>
using namespace std;

int main() {
    char name[] = "Komal";

    cout << name;
    return 0;
}
```

Key Points:

- Stored as a character array
 - Ends with '\0'
 - Requires manual handling for many operations
-

2. C++ String Class

C++ provides a built-in string class that is easier and safer to use.

Example:

```
#include <iostream>
#include <string>
using namespace std;

int main() {
```

```
string name = "Komal";

cout << name;
return 0;
}
```

Common String Operations

1. Input and Output

```
string name;
cin >> name;
cout << name;
```

2. String Length

```
string name = "Hello";
cout << name.length();
```

3. Concatenation (Joining Strings)

```
string a = "Hello";
string b = "World";

cout << a + " " + b;
```

4. Comparing Strings

```
string a = "apple";
string b = "banana";

if (a == b) {
    cout << "Equal";
} else {
    cout << "Not Equal";
}
```

5. Accessing Characters

```
string name = "Komal";
```

```
cout << name[0]; // K
```

6. Appending Strings

```
string name = "Komal";  
name.append(" Nafde");
```

```
cout << name;
```

19. How are arrays initialized in C++? Provide examples of both 1D and 2D arrays?

Ans- **Array Initialization in C++**

Array initialization in C++ means assigning values to an array at the time of declaration or after declaration. Arrays can be initialized in both **1D (single-dimensional)** and **2D (multi-dimensional)** forms.

1. Initialization of 1D Array

A 1D array stores elements in a single line (row).

Method 1: Initialization at Declaration

```
#include <iostream>  
using namespace std;  
  
int main() {  
    int arr[5] = {10, 20, 30, 40, 50};  
  
    cout << arr[2]; // Output: 30  
    return 0;  
}
```

Method 2: Partial Initialization

If fewer values are given, remaining elements become **0**.

```
int arr[5] = {10, 20};
```

Result:

10, 20, 0, 0, 0

Method 3: Without Size Specification

```
int arr[] = {1, 2, 3, 4};
```

Compiler automatically sets size = 4.

2. Initialization of 2D Array

A 2D array stores elements in **rows and columns (matrix form)**.

Method 1: Full Initialization

```
#include <iostream>
using namespace std;

int main() {
    int matrix[2][3] = {
        {1, 2, 3},
        {4, 5, 6}
    };

    cout << matrix[1][2]; // Output: 6
    return 0;
}
```

Method 2: Partial Initialization

```
int matrix[2][3] = {
    {1, 2},
```

```
{4}  
};
```

Result:

Row 1: 1 2 0

Row 2: 4 0 0

Method 3: Without Row Size

```
int matrix[][3] = {  
    {1, 2, 3},  
    {4, 5, 6}  
};
```

Number of columns must be specified.

20. Explain string operations and functions in C++?

Ans- **String Operations and Functions in C++**

In C++, strings are used to store and manipulate text. The **C++ string class** provides many built-in functions to perform operations easily.

String handling is available through the header:

```
#include <string>
```

1. String Input and Output

Example:

```
#include <iostream>  
#include <string>  
using namespace std;
```

```
int main() {  
    string name;  
  
    cin >> name;    // input  
    cout << name;   // output
```

```
    return 0;  
}
```

2. String Concatenation (Joining Strings)

Used to join two strings using + or append().

Example:

```
string a = "Hello";  
string b = "World";
```

```
cout << a + " " + b;
```

Using append():

```
string a = "Hello";  
a.append(" World");
```

```
cout << a;
```

3. Finding String Length

Used to get the number of characters.

Example:

```
string name = "Komal";  
cout << name.length();
```

4. Accessing Characters

Each character can be accessed using index.

Example:

```
string name = "Komal";
```

```
cout << name[0]; // K
```

5. Comparing Strings

Used to compare two strings using ==, <, >.

Example:

```
string a = "apple";  
string b = "banana";  
  
if (a == b) {  
    cout << "Equal";  
} else {  
    cout << "Not Equal";  
}
```

6. Substring Function

Used to extract part of a string.

Syntax:

```
string.substr(start_index, length);
```

Example:

```
string text = "Programming";  
  
cout << text.substr(0, 4); // Prog
```

7. Find Function

Used to find position of a substring.

Example:

```
string text = "Hello World";  
  
cout << text.find("World");
```

8. Replace Function

Used to replace part of a string.

Example:

```
string text = "I like C++";  
  
text.replace(7, 3, "Java");  
  
cout << text;
```

9. Insert Function

Used to insert text at a position.

Example:

```
string text = "Hello";  
  
text.insert(5, " World");  
  
cout << text;
```

10. Erase Function

Used to remove part of a string.

Example:

```
string text = "Hello World";  
  
text.erase(5, 6);  
  
cout << text;
```

21. Explain the key concepts of Object-Oriented Programming (OOP)?

Ans- **Key Concepts of Object-Oriented Programming (OOP)**

Object-Oriented Programming (OOP) is a programming approach based on **objects and classes**. It helps in organizing programs in a more structured and reusable way.

The main concepts of OOP are:

1. Class

A **class** is a blueprint or template for creating objects. It defines properties (data) and behaviors (functions).

Example:

```
class Student {  
public:  
    int id;  
    string name;  
  
    void show() {  
        cout << id << " " << name;  
    }  
};
```

2. Object

An **object** is an instance of a class. It represents a real-world entity.

Example:

```
Student s1;  
s1.id = 101;  
s1.name = "Komal";
```

3. Encapsulation

Encapsulation means **wrapping data and functions together** in a single unit (class) and restricting direct access to data.

Benefits:

- Data security
- Controlled access using functions

4. Abstraction

Abstraction means **hiding internal details and showing only essential features**.

Example:

When you use a mobile phone, you don't know internal working—only features are visible.

5. Inheritance

Inheritance allows one class to **acquire properties and behavior of another class**.

Example:

```
class Parent {
public:
    void show() {
        cout << "Parent class";
    }
};

class Child : public Parent {
};
```

6. Polymorphism

Polymorphism means **one name, many forms**. A function can behave differently in different situations.

Types:

- Compile-time (function overloading)
- Run-time (function overriding)

Example:

```
void add(int a, int b);
void add(float a, float b);
```

7. Dynamic Binding

In dynamic binding, the function call is resolved at **run time**, not compile time.

8. Message Passing

Objects communicate with each other by sending messages (calling functions)

22. What are classes and objects in C++? Provide an example.

Ans- **Classes and Objects in C++**

In C++, **classes and objects** are the basic concepts of Object-Oriented Programming (OOP).

1. Class

A **class** is a blueprint or template used to create objects. It defines:

- Data members (variables)
- Member functions (methods)

Example:

```
class Student {  
public:  
    int id;  
    string name;  
  
    void display() {  
        cout << id << " " << name;  
    }  
};
```

2. Object

An **object** is an instance of a class. It represents a real-world entity and uses the properties of a class.

Example:

```
Student s1;  
s1.id = 101;  
s1.name = "Komal";
```

Complete Example Program

```
#include <iostream>  
using namespace std;  
  
class Student {  
public:  
    int id;  
    string name;  
  
    void display() {  
        cout << "ID: " << id << endl;  
        cout << "Name: " << name << endl;  
    }  
};  
  
int main() {  
    Student s1; // object creation  
  
    s1.id = 101;  
    s1.name = "Komal";  
  
    s1.display(); // calling function using object  
  
    return 0;  
}
```

Output:

```
ID: 101  
Name: Komal
```

23. What is inheritance in C++? Explain with an example.

Ans- **Inheritance in C++**

Inheritance is an important feature of Object-Oriented Programming (OOP) in C++. It allows one class (child class) to **acquire properties and behaviors** of another class (parent class).

The main purpose of inheritance is **code reusability**.

Key Terms

- **Base Class (Parent Class):** The class whose properties are inherited.
 - **Derived Class (Child Class):** The class that inherits properties.
-

Syntax:

```
class BaseClass {  
    // members  
};  
  
class DerivedClass : public BaseClass {  
    // additional members  
};
```

Example of Inheritance in C++

```
#include <iostream>  
using namespace std;  
  
// Base class  
class Animal {  
public:  
    void eat() {  
        cout << "Eating..." << endl;  
    }  
};  
  
// Derived class  
class Dog : public Animal {  
public:
```

```
void bark() {  
    cout << "Barking..." << endl;  
}  
};  
  
int main() {  
    Dog d;  
  
    d.eat(); // inherited from Animal  
    d.bark(); // own function  
  
    return 0;  
}
```

Output:

Eating...
Barking...

Types of Inheritance in C++

1. **Single Inheritance** – one base class, one derived class
2. **Multiple Inheritance** – one class inherits from multiple classes
3. **Multilevel Inheritance** – class inherits from a derived class
4. **Hierarchical Inheritance** – multiple classes inherit from one base class
5. **Hybrid Inheritance** – combination of multiple types

Inheritance in C++

Inheritance is an important feature of Object-Oriented Programming (OOP) in C++. It allows one class (child class) to **acquire properties and behaviors** of another class (parent class).

The main purpose of inheritance is **code reusability**.

Key Terms

- **Base Class (Parent Class):** The class whose properties are inherited.
 - **Derived Class (Child Class):** The class that inherits properties.
-

Syntax:

```
class BaseClass {  
    // members  
};  
  
class DerivedClass : public BaseClass {  
    // additional members  
};
```

Example of Inheritance in C++

```
#include <iostream>  
using namespace std;  
  
// Base class  
class Animal {  
public:  
    void eat() {  
        cout << "Eating..." << endl;  
    }  
};  
  
// Derived class  
class Dog : public Animal {  
public:  
    void bark() {  
        cout << "Barking..." << endl;  
    }  
};  
  
int main() {  
    Dog d;
```

```
d.eat(); // inherited from Animal
d.bark(); // own function

return 0;
}
```

Output:

Eating...
Barking...

Types of Inheritance in C++

1. **Single Inheritance** – one base class, one derived class
2. **Multiple Inheritance** – one class inherits from multiple classes
3. **Multilevel Inheritance** – class inherits from a derived class
4. **Hierarchical Inheritance** – multiple classes inherit from one base class
5. **Hybrid Inheritance** – combination of multiple types

24. What is encapsulation in C++? How is it achieved in classes?

Ans- **Encapsulation in C++**

Encapsulation is one of the key concepts of Object-Oriented Programming (OOP) in C++. It means **wrapping data (variables) and functions (methods) together into a single unit called a class** and **restricting direct access to some of the data**.

In simple words:

👉 *Encapsulation = Data Hiding + Bundling of data and functions*

How Encapsulation is Achieved in C++

Encapsulation is achieved using:

- **Classes**
- **Access specifiers:**

- private
- public
- protected

1. private members

- Data is hidden from outside the class
- Cannot be accessed directly

2. public methods

- Used to access and modify private data safely

Example of Encapsulation in C++

```
#include <iostream>
using namespace std;

class Student {
private:
    int age; // hidden data

public:
    void setAge(int a) {
        age = a; // controlled access
    }

    void getAge() {
        cout << "Age = " << age;
    }
};

int main() {
    Student s;

    s.setAge(20); // setting value
    s.getAge(); // getting value
```

```
    return 0;  
}
```

Output:

Age = 20